50. Inference Serving: Batching, Queues, Sidecars

50. Inference Serving: Batching, Queues, Sidecars

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `50-inference-serving`

Learning objectives

- Queue inference requests
- Dynamic batch for GPU efficiency
- Use sidecar patterns for model runtimes
- Expose SLOs for latency vs throughput

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Inference serving needs queues and batching so GPUs/CPUs stay busy without melting under spikes.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Queue requests; workers pull batches.
- Max batch size and max wait time trade latency/throughput.
- Backpressure: reject or shed when queue is full.
- Timeouts return errors to clients.
- Sidecars often host models; Go handles orchestration.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-inference && cd /tmp/gw-inference
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

type Req struct{ ID int }

func CollectBatch(q []Req, max int) ([]Req, []Req) {
    if len(q) <= max {
        return q, nil
    }
    return q[:max], q[max:]
}

func main() {
    b, rest := CollectBatch([]Req{{1}, {2}, {3}}, 2)
    fmt.Println(len(b), len(rest))
}
```

Run it:

```
go run .
```

Expected output:

```
2 1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Implement CollectBatch.
2. Add a timer-based flush sketch.
3. Return an error when queue length exceeds capacity.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Serving shapes

- **Sync API**: simple, limited throughput
- **Queue + workers**: smooth spikes
- **Dynamic batching**: wait briefly to batch requests for GPU

Sidecar

Go app handles auth, quotas, routing; model runtime (Python/C++/TensorRT) runs beside it. Communicate via localhost gRPC/HTTP.

Tradeoffs

Batching raises latency p50 slightly to improve throughput — tune wait windows with metrics.

Failure modes to watch while coding

- One goroutine per GPU call unbounded
- No max queue depth (OOM)
- Ignoring tail latency

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Serving stacks (Triton etc.) — Go often fronts them.
- asyncio queues ≈ channel queues.
- Measure p99 latency, not only averages.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Load test with realistic prompt sizes — token counts drive cost.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Inference Serving: Batching, Queues, Sidecars” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- One goroutine per GPU call unbounded
- No max queue depth (OOM)

- Ignoring tail latency

Exercises

Work in `exercises/50-inference-serving/`. Read that folder's `README.md` before you code.

What you will build

Implement queue/batch helpers for inference stubs.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/50-inference-serving/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why dynamic batching?
2. What is a sidecar?
3. Queue depth purpose?

Further reading

- https://go.dev/blog/pipelines